We draw the line above the circles we already drew, and the line generator produces the plot shown in figure 4.18.

4.4.2 Drawing many lines with multiple generators

If we build a line constructor for each datatype in our set and call each with its own path, as shown in the following listing, then you can see the variation over time for each of your datapoints. Listing 4.10 demonstrates how to build those generators with our dataset, and figure 4.19 shows the results of that code.

Listing 4.10 Line generators for each tweetdata

```
var tweetLine = d3.svg.line()
    .x(function(d) {
        return xScale(d.day)
    })
    .y(function(d) {
        return yScale(d.tweets)
    });

var retweetLine = d3.svg.line()
    .x(function(d) {
        return xScale(d.day)
    })
    .y(function(d) {
        return yScale(d.retweets)
    });

var favLine = d3.svg.line()
    .x(function(d) {
        return xScale(d.day);
    })
    .y(function(d) {
        return yScale(d.favorites);
    });

d3.select("svg")
    .append("path")
    .attr("d", tweetLine(data))
    .attr("fill", "none")
    .attr("stroke", "darkred")
    .attr("stroke-width", 2);

d3.select("svg")
    .append("path")
    .attr("d", retweetLine(data))
    .attr("fill", "none")
    .attr("stroke", "gray")
    .attr("stroke-width", 3);

d3.select("svg")
    .append("path")
    .attr("d", favLine(data))
    .attr("fill", "none")
    .attr("stroke", "black")
    .attr("stroke-width", 2);
```

A more efficient way to do this would be to define one line generator, and then modify the .y() accessor on the fly as we call it for each line. But it's easier to see the functionality this way.

Notice how only the y accessor is different between each line generator.

Each line generator needs to be called by a corresponding new <path> element .